

第4次作业总结

题5

题目

Let $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_m)$ be two chains. Assume that in each chain, the nodes are in nondecreasing order of their data-field values. Write a C++ function to merge the two chains to obtain a new chain z in which the nodes are also in this order. Following the merge, x and y should represent empty chains because each node initially in x or y is now in z . No additional nodes may be used. What is the time complexity of your function?

参考答案

需要思考以下几个方面：

1. x链和y链初始是否为空，如果有为空怎么办？
2. x链和y链其中一条先结束，后面要如何处理？
3. 不能用额外的结点，所以z链没有自己的空间，只能移动x链和y链的指针。

可参考如下解答：

```
1 Chain Chain::Merge (Chain &x, Chain &y) {
2     Chain z;
3     ChainNode* current = nullptr;
4
5     // 处理初始情况
6     if (x.first == nullptr) {
7         z.first = y.first;
8         y.first = nullptr;
9         return z;
10    }
11    if (y.first == nullptr) {
12        z.first = x.first;
13        x.first = nullptr;
14        return z;
15    }
16
17    // 确定头节点
18    if (x.first->data <= y.first->data) {
```

```

19     z.first = x.first;
20     x.first = x.first->link;
21 } else {
22     z.first = y.first;
23     y.first = y.first->link;
24 }
25
26 current = z.first;
27
28 // 合并两个链表
29 while (x.first != nullptr && y.first != nullptr) {
30     if (x.first->data <= y.first->data) {
31         current->link = x.first;
32         x.first = x.first->link;
33     } else {
34         current->link = y.first;
35         y.first = y.first->link;
36     }
37     current = current->link; // 更新当前指针
38 }
39
40 // 连接剩余部分
41 if (x.first != nullptr) {
42     current->link = x.first;
43     x.first = nullptr; // 清空 x
44 } else {
45     current->link = y.first;
46     y.first = nullptr; // 清空 y
47 }
48
49 return z;
50 }

```

复杂度: $O(n + m)$

题6

题目

It is possible to traverse a chain in both directions (i.e., left to right and a restricted right-to-left traversal) by reversing the links during the left-to-right traversal. A possible configuration under this scheme is given in Figure 4.12. Assume that l and r are data members of Chain.

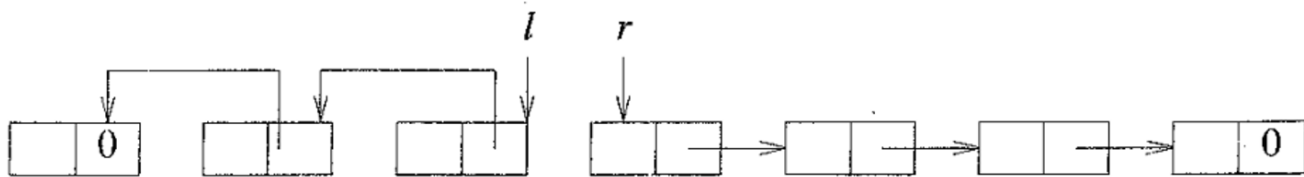


Figure 4.12: Possible configuration for a chain traversed in both directions

The pointer r points to the node currently being examined and l to the node on its left. Note that all nodes to the left of r have their links reversed.

(a) Write a C++ function to move pointer r , n nodes to the right from any given position (l, r) . If n is greater than the number of nodes to the right, set r to 0 and l to the rightmost node in the chain.

(b) Write a C++ function to move r , n nodes to the left from any given position (l, r) . If n is greater than the number of nodes to the left, set l to 0 and r to the leftmost node in the chain.

参考答案

从左向右遍历和从右向左遍历的过程中都需要翻转指针

可参考如下解答：

```

1 // (a)
2 void moveRight (ChainNode*& l, ChainNode*& r, int n) {
3 {
4     int ChainNode* saver = 0;
5     for (int i = 0; i < n; i++) {
6         if (r->next != 0) {
7             saver = r;
8             r = r->next;
9             saver->next = l;
10            l = saver;
11        }
12        else {
13            r->next = l;
14            l = r;
15            r = nullptr;
16            break;
17        }
18    }
19 }

```

```

20
21 // (b)
22 void moveLeft(ChainNode*& l, ChainNode*& r, int n) {
23 {
24     if(l == nullptr) return;
25     for (int i = 0; i < n; i++) {
26         ChainNode *node = l->next;
27         l->next = r;
28         r = l;
29         l = node;
30         if(l == nullptr) return;
31     }
32 }

```

题4

题目

Let $x_1, x_2, \dots, x_n$ be the elements of a chain. Each x_i is an integer. Write a C++ function to compute the expression $\sum_{i=1}^{n-5} (x_i * x_{i+5})$. [Hint: use two iterators to simultaneously traverse the chain.]

参考答案

需要考虑到：如果初始x链表长度小于5怎么办？

可参考如下解答：

```

1  template <typename T>
2  bool calculate(const chain<T>& chain, int& result){
3      typename Chain<T>::ChainIterator itr_front = chain.begin();
4      typename Chain<T>::ChainIterator itr_back = chain.begin();
5      result = 0;
6      //先判断front能不能指向第五个元素
7      int cnt = 0;
8      while( cnt < 5 && (itr_front!=chain.end())) {
9          itr_front++;
10         cnt++;
11     }
12     if ( itr_front == chain.end()) { //front不可以指向第五个(链表长度小于5)
13         return false;
14     }
15     while (itr_front) {
16         result += *itr_back * *itr_front;
17         itr_back++;

```

```
18         itr_front++;  
19     }  
20     return true;  
21 }
```